

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ижевский государственный технический университет имени М.Т. Калашникова»
(ФГБОУ ВО «ИжГТУ имени М.Т. Калашникова»)

Факультет/Институт _____

Кафедра _____

Направление подготовки (специальность) _____

Направленность (профиль/специализация/программа) подготовки _____

ОТЧЕТ ПО ПРАКТИКЕ

наименование типа и вида практики

полное наименование места прохождения практики и структурного подразделения

Выполнил обучающийся _____ / _____ /

подпись ФИО, курс, номер группы

Дата сдачи отчета: « _____ » _____ 202 ____ г.

Дата аттестации « _____ » _____ 202 ____ г.

Оценка _____

Руководитель практики от ИжГТУ имени М.Т. Калашникова

подпись И.О. Фамилия, должность, ученая степень

Заведующий кафедрой _____

(подпись) И.О. Фамилия, ученая степень

ОТЗЫВ о результатах прохождения практики обучающимся
на

(Ф. И. О. обучающегося)

Обучающийся _____ курса направления подготовки/специальности

09.03.01 «_____»

факультета/института

«_____»

в период прохождения практики

освещаются следующие вопросы: конкретные результаты, полученные обучающимся;
оценка степени освоения обучающимся теоретических знаний и практических навыков;
оценка сформированности компетенций; степень выполнения программы практики,
индивидуального задания, личный вклад практиканта (насколько самостоятельно
выполнялась работа), отметки о личностных качествах обучающегося, его отношении к
делу; рекомендации по совершенствованию профессиональной подготовки обучающихся.

Оценка результатов практики обучающегося _____

отлично/хорошо/удовлетворительно/зачтено

Руководитель практики _____ / _____ /

подпись И.О.Фамилия, должность, ученая степень

Содержание

Содержание	3
Введение	4
Техническое задание	5
Краткие сведения о решаемых задачах	8
Разработка программы	8
Результаты работы программы.....	14
Руководство пользователя.....	16
Заключение.....	20
Список литературы.....	21
Приложения.....	22

Введение

Актуальность задачи

В современном мире цифровые коммуникации стали неотъемлемой частью повседневной жизни. Разработка веб-приложения для обмена сообщениями обусловлена возрастающей потребностью в удобных средствах коммуникации, которые можно использовать в любой момент.

Цель работы

Создание полнофункционального веб-приложения с возможностью регистрации пользователей и обмена сообщениями в реальном времени.

Краткое содержание

В отчете описан процесс постановки задачи и технического задания, теоретические основы работы веб-приложения, этапы разработки программы, результаты тестирования, а также рекомендации по эксплуатации.

Техническое задание

1. Введение

1.1 Наименование программы

Chat

1.2 Назначение и область применения

Программа предназначена для обмена текстовыми сообщениями между пользователями в режиме реального времени.

2. Основания для разработки

2.1 Основания для разработки

Разработка программы осуществляется по заданию учебной практики.

2.2 Наименование и условное обозначение темы разработки

Веб-приложение для обмена текстовыми сообщениями.

3. Назначение разработки

Программа позволяет:

Регистрацию и авторизацию пользователей

Найти конкретного пользователя по его имени

Создать диалог с конкретным пользователем

Отправить сообщение в диалоге с другим пользователем

Обмен сообщениями в реальном времени

Отображение статуса "онлайн"

4. Требования к программе

4.1 Требования к функциональным характеристикам

Программа должна обеспечивать:

Регистрацию учетной записи и вход в нее

Создание диалога с конкретным пользователем

Отправку сообщения другому пользователю

Сохранение истории сообщений

Индикатор набора текста

4.2 Требования к надежности

Корректная обработка ошибок, связанных с отсутствием веб-страницы, доступом к ресурсам и работой сервера.

4.3 Условия эксплуатации

Температура внутри помещения: от +18 до +27 градусов Цельсия

Влажность воздуха: от 30% до 50%

4.4 Требование к составу и параметрам технических средств

Процессор: Любой 2-ядерный процессор x86/x64 с частотой 1.2 ГГц или выше

Оперативная память: 512 Мб (рекомендуется 1 Гб)

Свободное место на диске: 1 Гб (рекомендуется 4 Гб)

4.5 Требования к информационной и программной совместимости

Операционная система: Windows 10/11

Браузер: Chrome 137.0.0.0 или выше, Yandex 25.0.0.0 или выше.

Наличие .NET Framework 4.7.2 или выше

Наличие языка C# 7.2 или выше

Наличие SQL Server 2019+ или выше

5. Требования к программной документации

5.1 Состав программной документации

Техническое задание

Руководство пользователя

Исходный код программы

6. Технико-экономические показатели

Программа с открытым исходным кодом.

Минимальные требования к ресурсам:

Оперативная память: 512 Мб

Свободное место на диске: 1 Гб

Финансовые издержки: 0 рублей

7. Стадии и этапы разработки

7.1 Стадии разработки

1. Проектирование интерфейса (страницы сайта)
2. Backend-разработка, Frontend-разработка
3. Тестирование работы всех функций
4. Подготовка документации

7.2 Этапы разработки

Анализ требований

Разработка интерфейса

Кодирование

Отладка

Сдача программы

8. Порядок контроля и приемки

8.1. Виды испытаний

Функциональное тестирование создания учетной записи и входа в нее, создания нового диалога и отправки сообщений.

8.2 Общие требования к приемке

Программа считается готовой к использованию после успешного прохождения всех тестов.

Разработчик: Николаева Наталья Алексеевна

Дата: 19.06.2025

Краткие сведения о решаемых задачах

ASP.NET Core представляет собой современный фреймворк, предназначенный для создания веб-приложений на языке C#. Данный фреймворк характеризуется высокой производительностью и кроссплатформенностью, что сделало его идеальным выбором для серверной части приложения. ASP.NET Core обеспечивает встроенную поддержку веб-API, упрощает процесс развертывания и интеграции с различными сервисами.

Для хранения данных пользователей и истории сообщений использовалась **SQL Server** — надежная реляционная система управления базами данных от Microsoft. SQL Server гарантирует целостность и безопасность хранимых данных, что является важным аспектом для чат-приложений.

Для реализации функциональности в реальном времени была применена библиотека **SignalR**. Она позволяет мгновенно передавать сообщения между пользователями без необходимости обновления веб-страниц, что значительно улучшает пользовательский опыт.

Entity Framework Core выступает в роли ORM (объектно-реляционного сопоставления), обеспечивая удобный и эффективный способ работы с базой данных непосредственно из кода, написанного на C#.

Для аутентификации и авторизации пользователей использовалось решение **ASP.NET Core Identity**. Это встроенное решение позволяет управлять учетными записями пользователей, паролями и ролями, гарантируя высокий уровень безопасности.

На клиентской стороне используются стандартные веб-технологии: **HTML** для структурирования страниц, **CSS** для стилизации интерфейса и **JavaScript** для разработки интерактивных элементов.

Сочетание приведенных технологий обеспечивает создание полнофункционального, безопасного и производительного чат-приложения с возможностями регистрации пользователей, обмена сообщениями в реальном времени, а также хранения истории переписки.

Разработка программы

1. Обоснование выбора проектных решений

В процессе разработки чат-приложения был выбран язык программирования C# версии 13.0 и платформа .NET 9.0, что связано с рядом ключевых преимуществ данной технологической связки.

C# 13.0 обеспечивает улучшенную производительность и расширенные возможности, такие как усовершенствованные шаблоны сопоставления, оптимизация работы с асинхронными операциями и лучшая работа с памятью. Эти характеристики критичны для чат-приложения, где важна скорость обработки сообщений и эффективное управление ресурсами.

Платформа .NET 9.0 выделяется кроссплатформенной поддержкой, что позволяет развертывать приложение не только на Windows, но и на других операционных системах. Она также предоставляет улучшенные системы безопасности и высокую производительность работы с сетевыми соединениями, что важно для обеспечения качественной работы чата в реальном времени.

Наличие встроенной поддержки SignalR в .NET 9.0 позволяет реализовать двусторонний обмен сообщениями без необходимости использовать сторонние библиотеки.

В сравнении с альтернативами, такими как Java, Python или Node.js, выбранная связка демонстрирует лучшую производительность в сетевых соединениях и предсказуемое управление памятью. Экосистема .NET предлагает все необходимые инструменты для работы с базами данных, аутентификации пользователей и развертывания приложения, что значительно ускоряет процесс разработки.

2. Описание наиболее важных алгоритмов

Алгоритм регистрации:

1. Пользователь придумывает своё имя пользователя, вводит Email и придумывает пароль
2. Если имя пользователя или Email уже есть в базе, то показывает ошибку, если пароль слишком простой, то показывает требования для пароля
3. Если регистрация успешная, то введенный пароль шифруется, а затем все данные сохраняются в базе SQL, создается новая учетная запись
4. Пользователь автоматически входит в систему и попадает в чат

Схема алгоритма представлена на рисунке 1

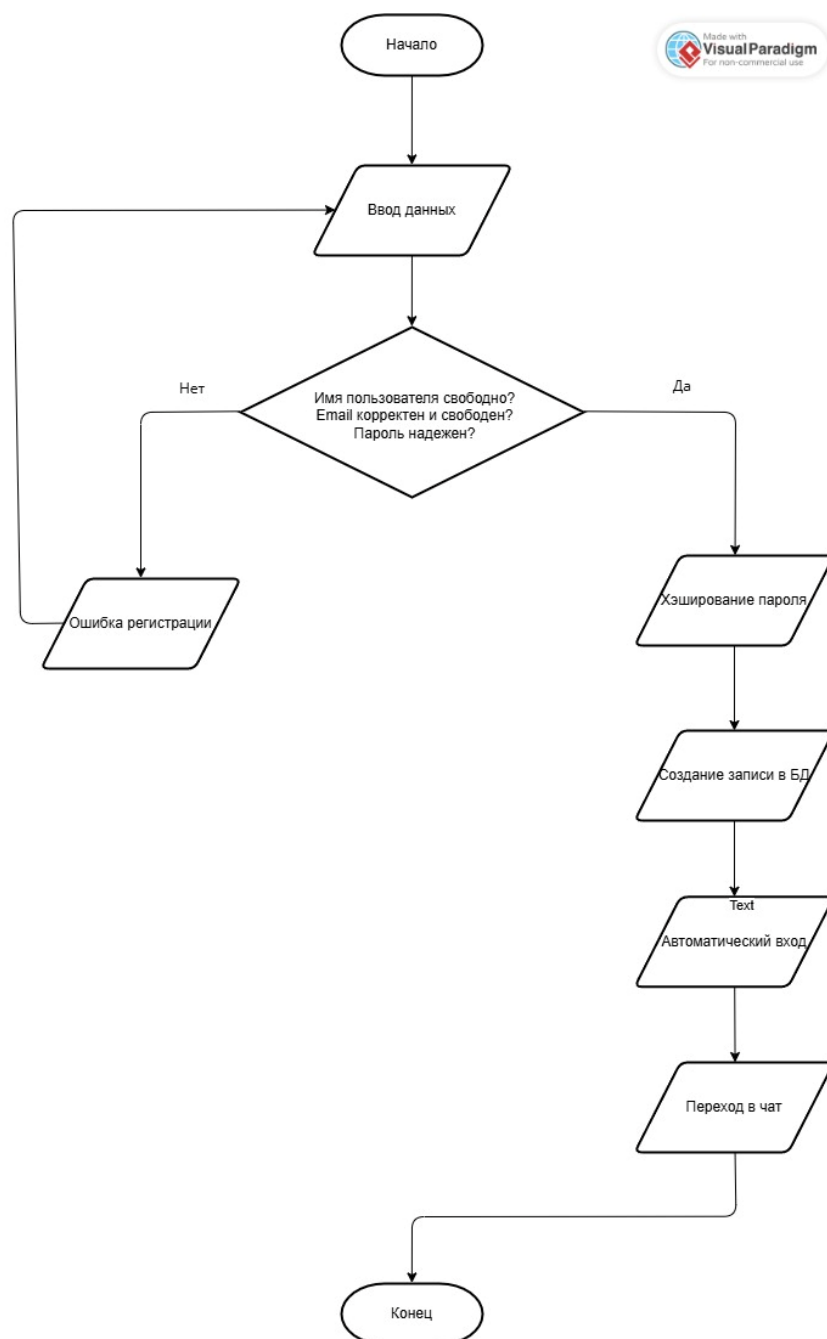


Рисунок 1 – Схема алгоритма регистрации

Алгоритм входа:

1. Пользователь вводит Email и пароль
2. Сервер проверяет есть ли пользователь с таким Email в базе SQL, далее проверяет хэш, введенного пароля, с тем хэшем, который уже сохранен в базе SQL
3. Если пользователи существуют и хэши паролей совпадают, то сервер перенаправляет на главную страницу чата, иначе показывает ошибку.

Схема алгоритма представлена на рисунке 2

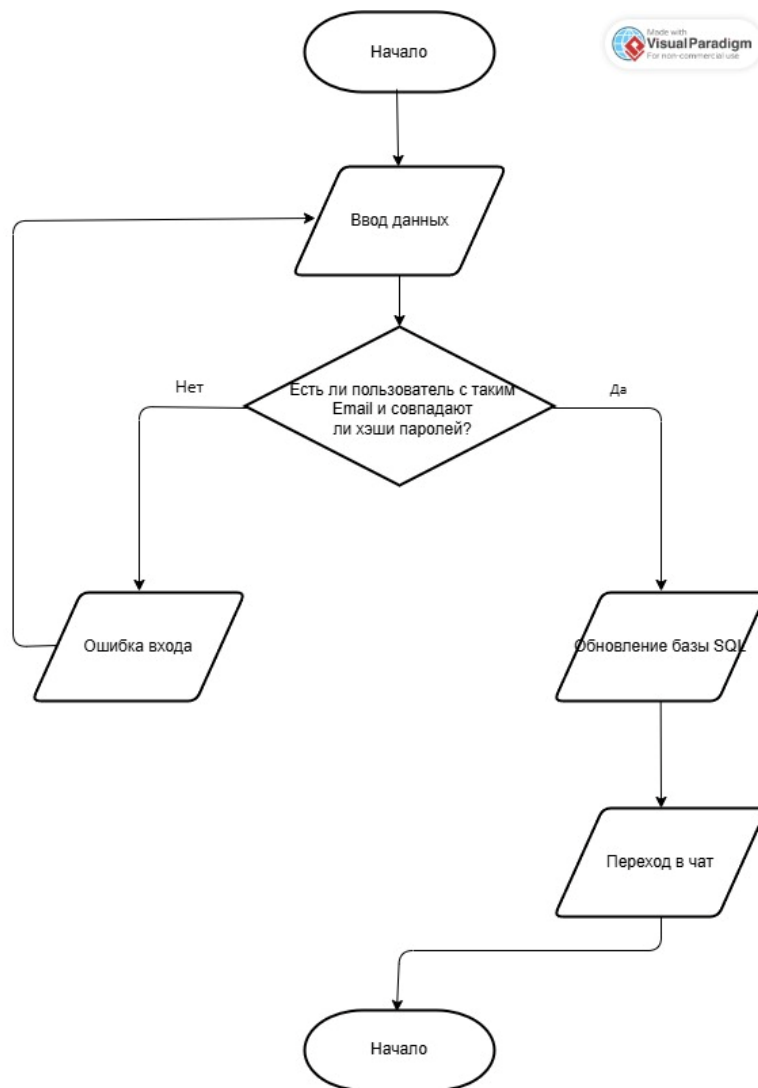


Рисунок 2 - Схема алгоритма входа

Алгоритм отправки сообщений:

1. На странице диалога есть форма, в которой содержатся два поля: первое – скрытое, оно отвечает за то, кому мы отправляем сообщение, второе – содержит в себе сообщение. Также форма содержит кнопку «Отправить», которая отвечает за отправку данных серверу
2. Пользователь вводит сообщение, нажимает кнопку «Отправить»
3. Данные отправляются серверу
4. Если сообщение содержит не только символы пробела, то сервер сохраняет в базе SQL, с указанием отправителя, получателя, текста сообщения и времени отправки, и отправляет сообщение в реальном времени, иначе сообщение игнорируется, и пользователь остается на той же странице чата

Схема алгоритма представлена на рисунке 3

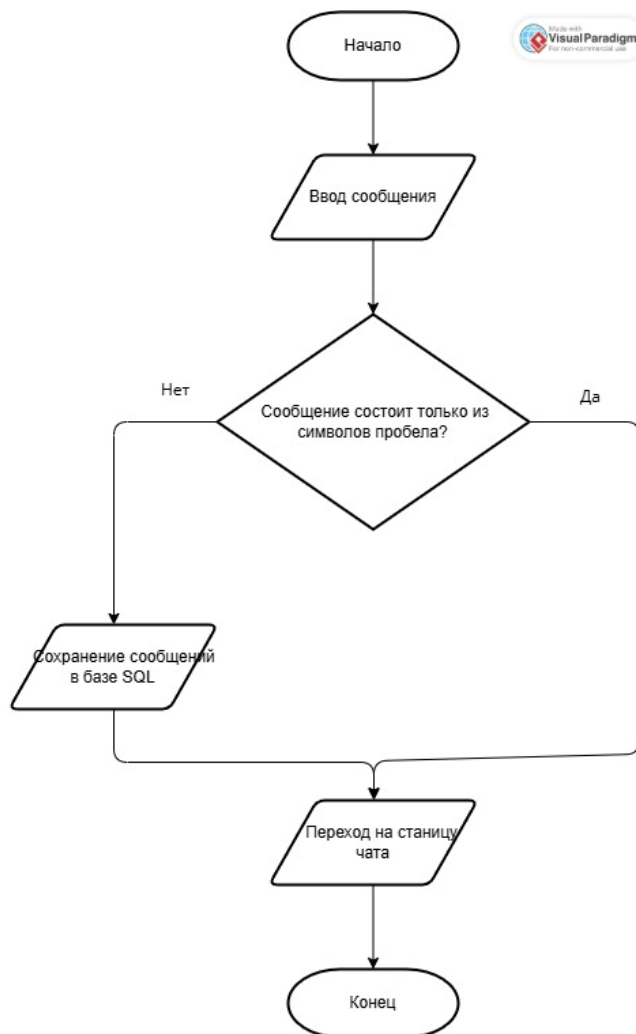


Рисунок 3 – Схема алгоритма отправки сообщения

Алгоритм обработки SignalR:

1. Когда пользователь открывает чат, браузер устанавливает соединение с сервером через WebSocket. Сервер принимает подключение и запоминает пользователя
2. SignalR объединяет пользователей в виртуальные «комнаты» (группы). Например, для диалога между User1 и User2 создается группа
3. Клиент отправляет сообщение, сервер получает его и пересылает всем в группе (диалоге), получатель видит сообщение

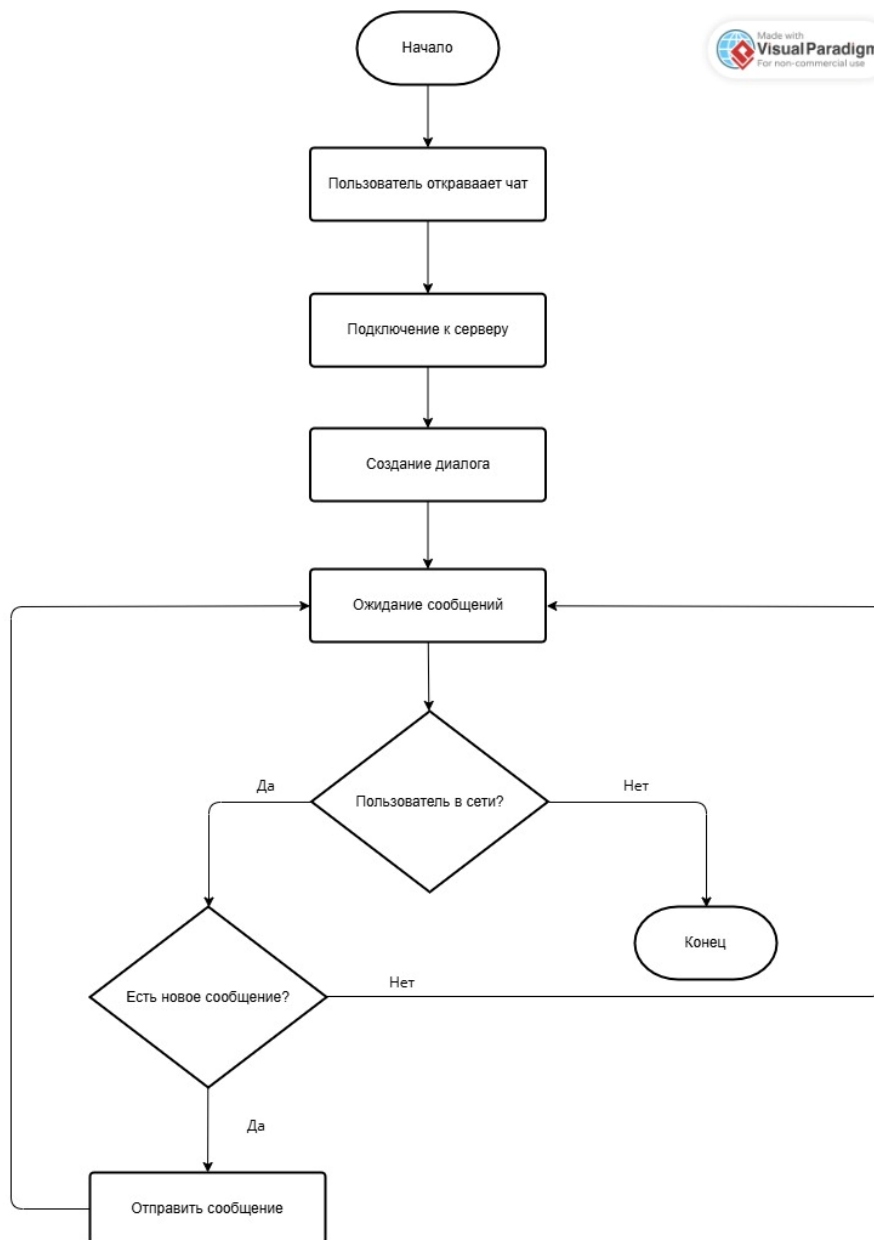


Рисунок 4 – Схема алгоритма обработки SignalR

Описание структуры файла users.cs

Этот файл содержит учетные данные, профильную информацию, и обеспечивает связь с сообщениями:

1. DisplayName – отображаемое имя
2. RegistrationDate– дата регистрации
3. LastOnline – последний раз в сети
4. IsOnline – онлайн статус
5. AvatarUrl – аватар URL
6. ConnectionId – временный ID подключения SignalR
7. SentMessages – отправленные сообщения
8. ReceivedMessages – полученные сообщения

Описание структуры файла Message.cs

Файл существует для хранения текста сообщений, факта прочтения, связывает отправителя и получателя :

1. Id – уникальный номер диалога
2. SenderId – отправитель
3. ReceiverId – получатель
4. Content – текст сообщения
5. SentTime – время отправки
6. IsRead – статус сообщения
7. ReadTime – время прочтения
8. Sender – ссылка на отправителя
9. Receiver – ссылка на получателя

Результаты работы программы

Запускаем программу и ждем загрузки главной страницы сайта, которая изображена на рисунке 5.

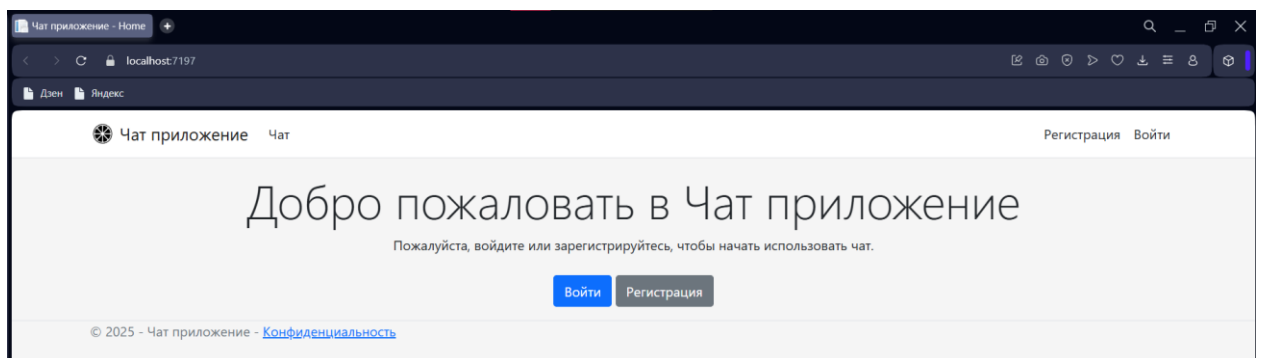


Рисунок 5 – Главная страница сайта

Если нажать на кнопку «Войти», то перейдем на страницу входа, которая изображена на рисунке 6.

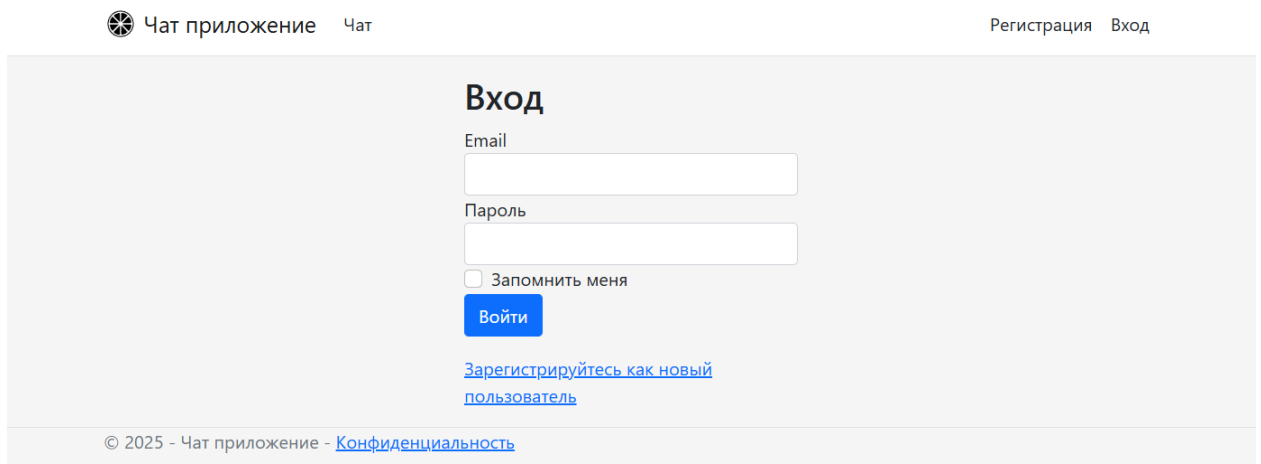


Рисунок 6 – Страница входа

Если нажать на кнопку «Регистрация», то перейдем на страницу регистрации, которая изображена на рисунке 7.

Чат приложение Чат

Регистрация Вход

Регистрация

Отображаемое имя

Email

Пароль

Повторите пароль

Зарегистрироваться

© 2025 - Чат приложение - [Конфиденциальность](#)

Рисунок 7 – Страница регистрации

Со страницы входа мы можем перейти по ссылке «Зарегистрируйтесь как новый пользователь» на страницу регистрации, которая изображена на рисунке 8.

Чат приложение Чат

Регистрация Вход

Регистрация

Отображаемое имя

Email

Пароль

Повторите пароль

Зарегистрироваться

© 2025 - Чат приложение - [Конфиденциальность](#)

Рисунок 8 – Страница регистрации

Страница, отображающая все Ваши диалоги, показана на рисунке 9.

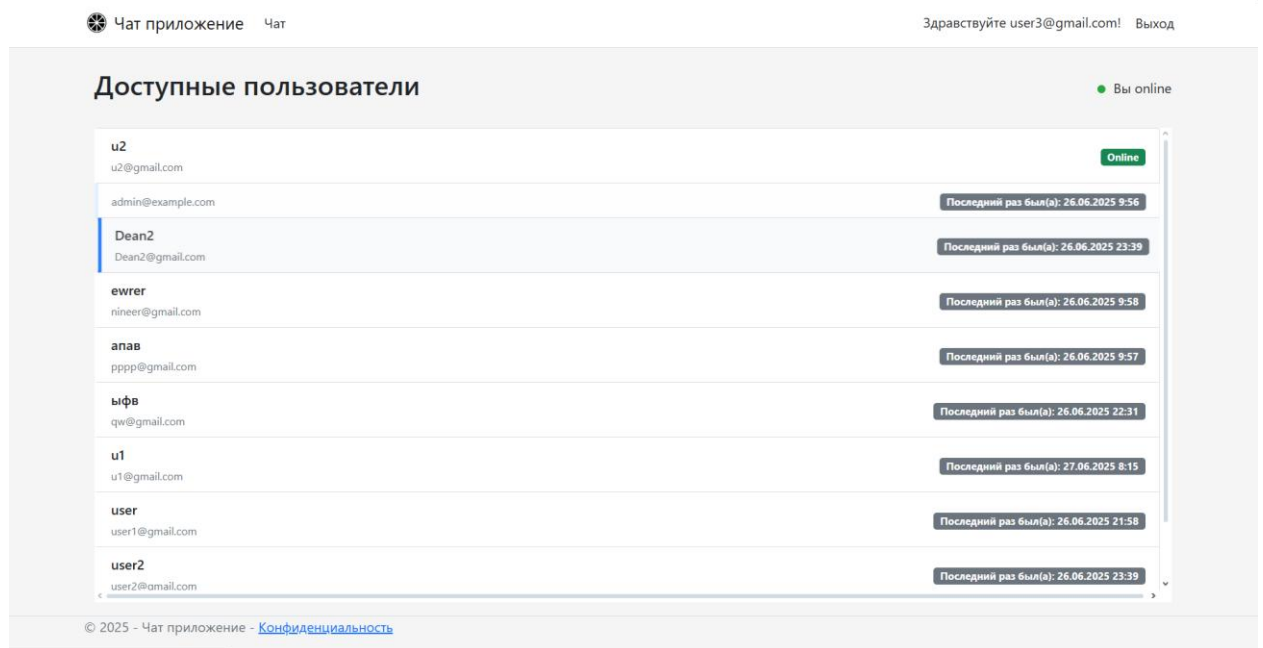


Рисунок 9 – Страница со всеми диалогами

Страница диалога с конкретным пользователем показана на рисунке 10.

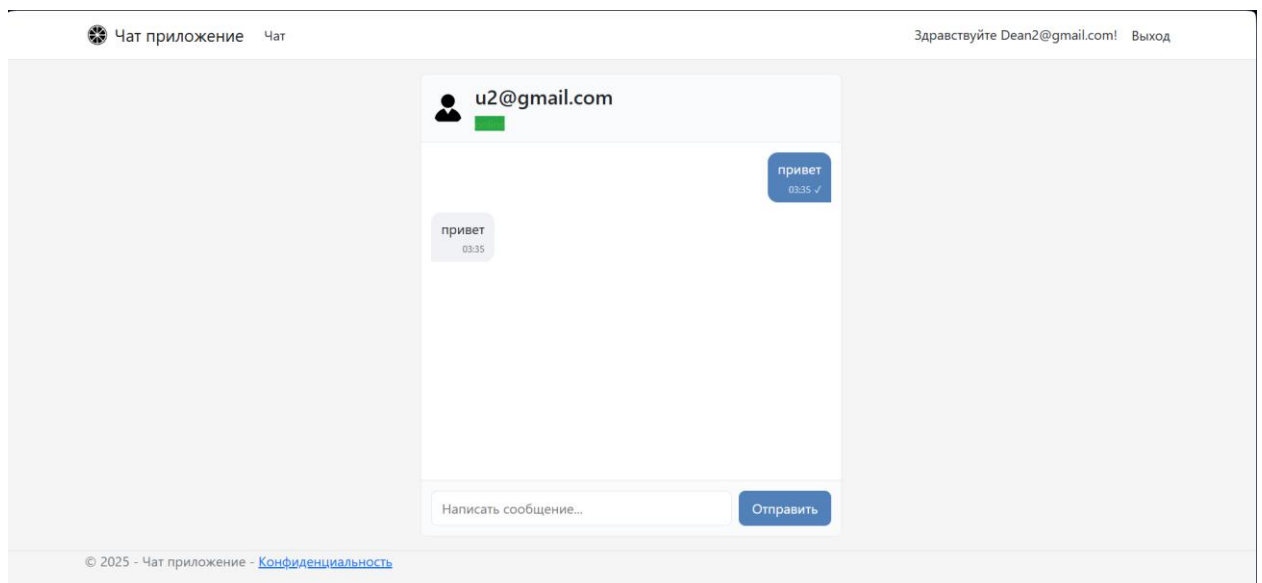


Рисунок 10 – Страница диалога

Руководство пользователя

Назначение программы

Программа предназначена для обмена текстовыми сообщениями и дает пользователю следующие возможности:

1. Создание учетной записи и вход в нее
2. Поиск конкретного пользователя по его имени
3. Создание с конкретным пользователем диалога
4. Написание текстовых сообщений

5. Статус пользователя

Условия выполнения программы

Веб-приложение используется:

В операционной системе Windows 10/11

В браузерах Chrome, Yandex, Firefox и др.

Выполнение программы

Запустите программу, откроется браузер и страница по ссылке <https://localhost:7197>.

У Вас загрузится главная страница. В верхней части и по середине есть строка навигации, которая включает в себя ссылку на Главную страницу, ссылку на страницу регистрации и ссылку на страницу входа.

Вход в учетную запись

На странице входа, которая изображена на рисунке 6, есть два поля:

1. Первое – для вашего Email пользователя
2. Второе – для вашего пароля

Под формой входа, изображенной на рисунке 11, есть ссылка на страницу регистрации.

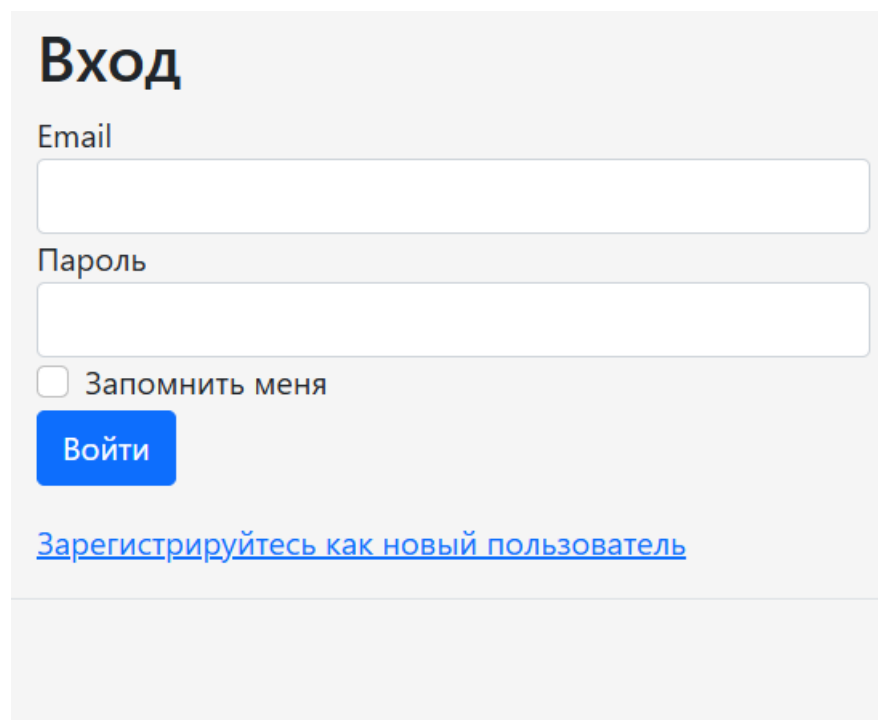


Рисунок 11 – Форма входа в учетную запись

Регистрация учетной записи

На странице регистрации есть форма в соответствии с рисунком 12, в которую нужно ввести соответствующие данные, а далее нажать кнопку «Зарегистрироваться».

Регистрация

Отображаемое имя

Email

Пароль

Повторите пароль

[Зарегистрироваться](#)

Рисунок 12 – Форма регистрации

Страница, на которой изображены все диалоги

На странице отображены все ссылки на диалоги, которые Вы начали ранее, по нажатию на которые Вы перейдете в диалог с конкретным пользователем и увидите статус пользователей, а сверху справа кнопка «Выход», которая позволяет выйти из аккаунта. Данная страница изображена на рисунке 13.

Чат приложение
Чат

Здравствуйте Dean2@gmail.com! [Выход](#)

Доступные пользователи

● Вы online

admin@example.com	Последний раз был(а): 26.06.2025 9:56
ewrgr nineer@gmail.com	Последний раз был(а): 26.06.2025 9:58
апаав pppp@gmail.com	Последний раз был(а): 26.06.2025 9:57
ыфв qw@gmail.com	Последний раз был(а): 26.06.2025 22:31
u1 u1@gmail.com	Последний раз был(а): 27.06.2025 8:15
u2 u2@gmail.com	Последний раз был(а): 01.07.2025 3:36
user user1@gmail.com	Последний раз был(а): 01.07.2025 3:29
user2 user2@gmail.com	Последний раз был(а): 26.06.2025 23:39
user3 user3@gmail.com	Последний раз был(а): 01.07.2025 3:32

Рисунок 13 – Страница, отображающая все диалоги

Страница диалога

На странице диалога, изображенной на рисунке 14, кнопка «Отправить» отвечает за отправку Вашего сообщения другому пользователю, сверху, Email пользователя Внизу страницы расположена форма сообщения. Сообщение вводится в поле ввода «Написать сообщение...», далее, чтобы отправить сообщение нужно нажать кнопку «Отправить».

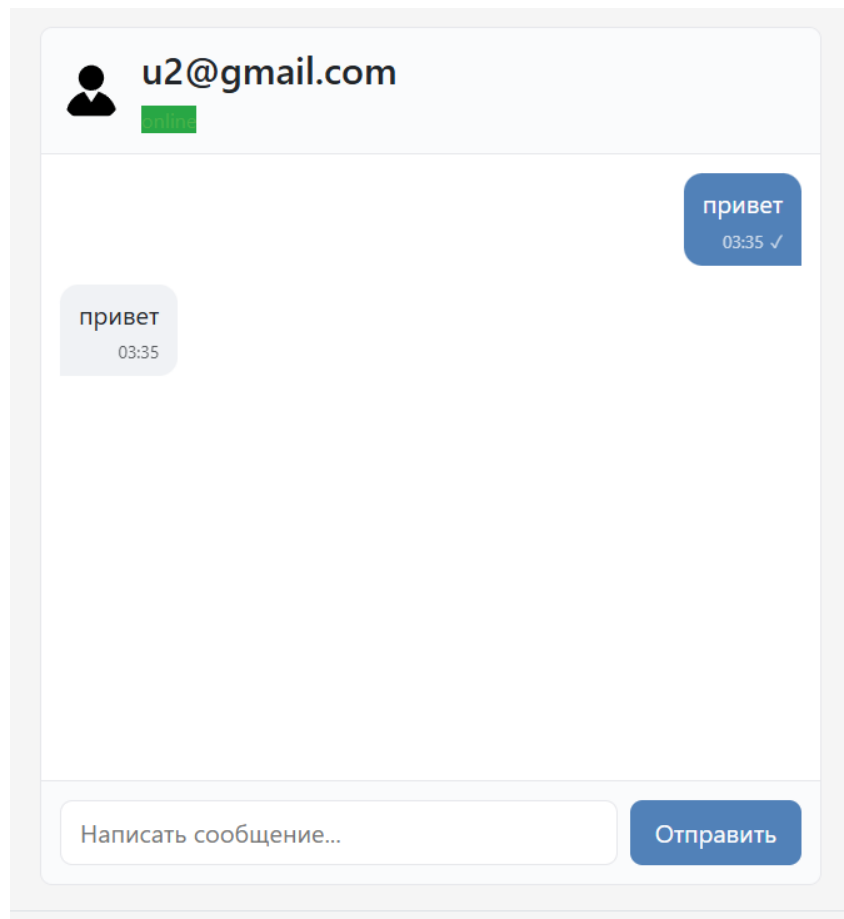


Рисунок 14 – Страница диалога

Заключение

В ходе выполнения учебной практики было разработано веб-приложение для обмена текстовыми сообщениями. Реализованы ключевые функции: регистрация и вход, обмен сообщениями в реальном времени, списки пользователей, история сообщений. Программа прошла успешное тестирование на наличие ошибок разного рода. Разработанная программа соответствует поставленным задачам и готова к практическому применению.

Список литературы

1. metanit.com Сетевое программирование в C# и .NET. [Электронный ресурс]. –URL: <https://metanit.com/sharp/net/>. (дата обращения 05.06.25)
2. thoughtbot.com Using HTTPListener to build a HTTP Server in C#. [Электронный ресурс]. –URL: <https://thoughtbot.com/blog/using-httplistener-to-build-a-http-server-in-csharp>. (дата обращения 11.06.25)
3. learn.microsoft.com SignalR Guide Класс. [Электронный ресурс]. –URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction?view=aspnetcore-9.0>. (дата обращения 12.06.25)
4. learn.microsoft.com Класс ASP.NET Core Documentation [Электронный ресурс]. –URL: <https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-9.0>. (дата обращения 15.06.25)
5. learn.microsoft.com Класс Entity Framework Core [Электронный ресурс]. –URL: <https://learn.microsoft.com/en-us/ef/core/>. (дата обращения 15.06.25)

Приложения

Код программы Program.cs

```
using Chat.Data;
using Chat.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.SqlServer;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
);
builder.Services.AddIdentity<User, IdentityRole<int>>()
    .AddEntityFrameworkStores<AppDbContext>
    .AddDefaultTokenProviders();
builder.Services.AddSignalR(options => {
    options.EnableDetailedErrors = builder.Environment.IsDevelopment();
    options.ClientTimeoutInterval = TimeSpan.FromMinutes(1);
});
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", builder =>
        builder.AllowAnyOrigin()
        .AllowAnyMethod()
        .AllowAnyHeader());
});
builder.Services.AddControllersWithViews();
var app = builder.Build();
app.UseHttpsRedirection(); // Перенаправление HTTP на HTTPS
app.UseStaticFiles();     // Поддержка статических файлов (wwwroot)
app.UseRouting();         // Маршрутизация запросов
app.UseAuthentication();  // Аутентификация пользователей
app.UseAuthorization();   // Авторизация доступа
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapHub<ChatHub>("/chatHub", options =>
{
    options.Transports =
Microsoft.AspNetCore.Http.Connections.HttpTransportType.WebSockets;
});
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    try
    {
        {
            var context = services.GetRequiredService<AppDbContext>();
            await context.Database.MigrateAsync();
            var userManager = services.GetRequiredService<UserManager<User>>();
            var roleManager =
services.GetRequiredService<RoleManager<IdentityRole<int>>>());
```

```

        await SeedData.Initialize(services, userManager, roleManager);
    }
    catch (Exception ex)
    {
        var logger = services.GetRequiredService<ILogger<Program>>();
        logger.LogError(ex, "Migration error");
    }
}
app.UseCors("AllowAll");
await app.RunAsync();

```

Код из файла **_LoginPartial.cshtml**

```

@using Microsoft.AspNetCore.Identity
@inject SignInManager<User> SignInManager
@inject UserManager<User> UserManager
<ul class="navbar-nav">
    @if (SignInManager.IsSignedIn(User))
    {
        <li class="nav-item">
            <a class="nav-link text-dark" asp-controller="Account" asp-action="Manage"
title="Manage">
                Здравствуйте @User.Identity?.Name!
            </a>
        </li>
        <li class="nav-item">
            <form class="form-inline" asp-controller="Account" asp-action="Logout" asp-
route-returnUrl="@Url.Action("Index", "Home")">
                <button type="submit" class="nav-link btn btn-link text-
dark">Выход</button>
            </form>
        </li>
    }
    else
    {
        <li class="nav-item">
            <a class="nav-link text-dark" asp-controller="Account" asp-
action="Register">Регистрация</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" asp-controller="Account" asp-
action="Login">Войти</a>
        </li>
    }
</ul>

```

Код из файла **_Layout.cshtml**

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Чат приложение - @ViewData["Title"]</title>
    <link rel="icon" href="~/favicon.ico" type="image/x-icon" />

```

```

<link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
<link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
@await RenderSectionAsync("Styles", required: false)
</head>
<body>
<header>
<nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white
border-bottom box-shadow mb-3">
<div class="container">
<a class="navbar-brand" asp-area="" asp-controller="Home" asp-
action="Index">

Чат приложение
</a>
<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-
bs-target="#navbarCollapse"
aria-expanded="false" aria-label="Переключить навигацию">
<span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarCollapse">
<ul class="navbar-nav me-auto">
<li class="nav-item">
<a class="nav-link text-dark" asp-controller="Chat" asp-
action="Index">Чат</a>
</li>
</ul>
<partial name="_LoginPartial" />
</div>
</div>
</nav>
</header>
<div class="container">
<main role="main" class="pb-3">
@if (TempData["Success"] != null)
{
<div class="alert alert-success alert-dismissible fade show" role="alert">
@TempData["Success"]
<button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
</div>
}
@if (TempData["Error"] != null)
{
<div class="alert alert-danger alert-dismissible fade show" role="alert">
@TempData["Error"]
<button type="button" class="btn-close" data-bs-dismiss="alert" aria-
label="Close"></button>
</div>
}
@RenderBody()
</main>

```



```

</div>
<footer class="border-top footer text-muted">
  <div class="container">
    &copy; @DateTime.Now.Year - Чат приложение - <a asp-area="" asp-
controller="Home" asp-action="Privacy">Конфиденциальность</a>
  </div>
</footer>
<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/signalr/dist/browser/signalr.min.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>

```

Код из файла Home/Index.cshtml

```

@{
  ViewData["Title"] = "Home";
}

<div class="text-center">
  <h1 class="display-4">Добро пожаловать в Чат приложение</h1>

  @if (User.Identity?.IsAuthenticated == true)
  {
    <p>Начните общаться в чате с другими пользователями!</p>
    <a asp-controller="Chat" asp-action="Index" class="btn btn-primary">Перейти в
чат</a>
  }
  else
  {
    <p>Пожалуйста, войдите или зарегистрируйтесь, чтобы начать использовать
чат.</p>
    <div class="mt-4">
      <a asp-controller="Account" asp-action="Login" class="btn btn-primary mr-
3">Вход</a>
      <a asp-controller="Account" asp-action="Register" class="btn btn-
secondary">Регистрация</a>
    </div>
  }
</div>

```

Код из файла Chat/Index.cshtml

```

@model List<User>

<div class="container mt-4">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <h2>Доступные пользователи</h2>
    <div class="online-status">
      <span class="dot online"></span>
      <span>Вы online</span>
    </div>
  </div>

```

```

    </div>
</div>
<div class="list-group users-list">
    @foreach (var user in Model)
    {
        <a href="@Url.Action("Conversation", "Chat", new { userId = user.Id })"
            class="list-group-item list-group-item-action user-item"
            data-user-id="@user.Id">
            <div class="d-flex justify-content-between align-items-center">
                <div class="user-info">
                    <div class="user-name">@user.DisplayName</div>
                    <small class="text-muted user-email">@user.Email</small>
                </div>
                <div class="user-status">
                    <span class="badge @(user.IsOnline ? "bg-success" : "bg-secondary")">
                        @(user.IsOnline ? "Online" : $"Последний раз был(а):
{user.LastOnline.ToString("g")}")
                    </span>
                </div>
            </div>
        </a>
    }
</div>
</div>
@section Styles {
    <style>
        .user-item {
            transition: all 0.3s ease;
            border-left: 4px solid transparent;
        }
        .user-item:hover {
            background-color: #f8f9fa;
            border-left-color: #0d6efd;
        }
        .user-name {

```

```

        font-weight: 500;
    }
    .user-email {
        font-size: 0.8rem;
    }
    .online-status .dot {
        height: 10px;
        width: 10px;
        border-radius: 50%;
        display: inline-block;
        margin-right: 5px;
    }
    .online-status .online {
        background-color: #28a745;
    }
    .users-list {
        max-height: 70vh;
        overflow-y: auto;
    }
</style>
}

@section Scripts {
<script>
    // Real-time status updates
    const connection = new signalR.HubConnectionBuilder()
        .withUrl("/chatHub")
        .configureLogging(signalR.LogLevel.Information)
        .build();
    connection.on("UserOnlineStatusChanged", (userId, isOnline) => {
        const badge = document.querySelector(.user-item[data-user-id="{userId}"]
.badge);
        if (badge) {
            badge.className = isOnline ? "badge bg-success" : "badge bg-secondary";
            badge.textContent = isOnline ? "Online" : "Offline";
        }
    });
}

```

```

    });
    connection.start()
      .then(() => console.log("Connected to status hub"))
      .catch(err => console.error("Connection failed: ", err));
    // Highlight active conversation
    document.querySelectorAll('.user-item').forEach(item => {
      item.addEventListener('click', function() {
        document.querySelectorAll('.user-item').forEach(i =>
          i.classList.remove('active'));
        this.classList.add('active');
      });
    });
  </script>
}

```

Код из файла Conversation.cshtml

```

@model ChatViewModel

<div class="vk-style-chat"
  data-current-user-id="@Model.CurrentUser.Id"
  data-other-user-id="@Model.OtherUser.Id">
  <div class="chat-header">
    <div class="user-info">
      
    </div>
    <h4>@Model.OtherUser.UserName</h4>
    <div class="status" id="userStatus">
      @if (Model.OtherUser.IsOnline)
      {
        <span class="online">online</span>
      }
      else
      {
        <span>Последний раз был(а)
        @Model.OtherUser.LastOnline.ToString("g")</span>

```

```

        }
    </div>
</div>
</div>
</div>
<div class="messages" id="messagesList">
    @foreach (var message in Model.Messages)
    {
        <div class="message @(message.SenderId == Model.CurrentUser.Id ? "out" : "in")"
            data-message-id="@message.Id">
            <div class="message-content">@message.Content</div>
            <div class="message-time">
                @message.SentTime.ToString("HH:mm")
                @if (message.SenderId == Model.CurrentUser.Id)
                {
                    <span class="status-icon @(message.IsRead ? "read" : "")">✓</span>
                }
            </div>
        </div>
    }
</div>
<div class="message-input">
    <input type="text" id="messageInput" placeholder="Написать сообщение...">
    <button id="sendButton">Отправить</button>
</div>
<div id="typingIndicator" style="display: none; padding: 10px; font-style: italic;">
    @Model.OtherUser.UserName печатает...
</div>
</div>
@section Styles {
    <style>

```

```

.vk-style-chat {
    max-width: 500px;
    margin: 0 auto;
    border: 1px solid #e7e8ec;
    border-radius: 8px;
    overflow: hidden;
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}

.chat-header {
    padding: 12px;
    background: #fafbfc;
    border-bottom: 1px solid #e7e8ec;

.user-info {
    display: flex;
    align-items: center;
}

.avatar {
    width: 40px;
    height: 40px;
    border-radius: 50%;
    margin-right: 12px;
}

.status {
    font-size: 13px;
    color: #656565;
}

.online {
    color: #4bb34b;
}

.messages {

```

```

height: 400px;
overflow-y: auto;
padding: 12px;
background: #fff;
display: flex;
flex-direction: column;
}
.message {
  max-width: 70%;
  margin-bottom: 12px;
  padding: 8px 12px;
  border-radius: 12px;
  line-height: 1.4;
  position: relative;
}
.message.in {
  background: #f0f2f5;
  align-self: flex-start;
  border-bottom-left-radius: 0;

.message.out {
  background: #5181b8;
  color: white;
  align-self: flex-end;
  border-bottom-right-radius: 0;
}
.message-content {
  word-wrap: break-word;
}
.message-time {
  font-size: 11px;

```

```

    text-align: right;
    opacity: 0.7;
    margin-top: 4px;
    display: flex;
    align-items: center;
    justify-content: flex-end;
}

.status-icon {
    margin-left: 4px;
}

.status-icon.read {
    color: #4bb34b;
}

.message-input {
    display: flex;
    padding: 12px;
    background: #fafbfc;
    border-top: 1px solid #e7e8ec;
}

#messageInput {
    flex: 1;
    padding: 8px 12px;
    border: 1px solid #e7e8ec;
    border-radius: 8px;
    outline: none;
}

#sendButton {
    margin-left: 8px;
    padding: 8px 16px;
    background: #5181b8;
    color: white;

```



```

border: none;
border-radius: 8px;
cursor: pointer;
transition: background-color 0.2s;
}

#sendButton:hover {
    background-color: #3a6ea5;
}

#sendButton:disabled {
    background-color: #cccccc;
    cursor: not-allowed;
}
</style>
}

@section Scripts {
    <script src="https://cdnjs.cloudflare.com/ajax/libs/microsoft-
    signalr/9.0.5/signalr.min.js"></script>

    <script>
        document.addEventListener('DOMContentLoaded', function() {
            const state = {
                isConnected: false,
                isSending: false,
                connectionAttempts: 0,
                MAX_RETRY_ATTEMPTS: 5
            };

            const chatContainer = document.querySelector('.vk-style-chat');
            const currentUserId = chatContainer.dataset.currentUserId;
            const otherUserId = chatContainer.dataset.otherUserId;
            const messageInput = document.getElementById('messageInput');
            const sendButton = document.getElementById('sendButton');
            const messagesList = document.getElementById('messagesList');
            const userStatus = document.getElementById('userStatus');

```

```

const typingIndicator = document.getElementById('typingIndicator');
const connection = new signalR.HubConnectionBuilder()
    .withUrl("/chatHub", {
        skipNegotiation: true,
        transport: signalR.HttpTransportType.WebSockets
    })
    .configureLogging(signalR.LogLevel.Warning)
    .withAutomaticReconnect({
        nextRetryDelayInMilliseconds: retryContext => {
            if (retryContext.elapsedMilliseconds < 60000) {
                return Math.random() * 2000 + 2000;
            }
            return null;
        }
    })
    .build();

connection.on("ReceiveMessage", (message) => {
    addMessage(message, message.senderId === currentUserId);
});

connection.on("MessageSent", (message) => {
    updateMessageStatus(message.id);
});

connection.on("UserTyping", () => {
    showTypingIndicator(true);
});

connection.on("UserStatusChanged", (status) => {
    updateUserStatus(status);
});

// Функции для работы с UI

function addMessage(message, isOutgoing) {
    const existingMessage = document.querySelector(`[data-message-id="${message.id}"]`);

```

```

    if (existingMessage) return;

    const messageElement = document.createElement('div');
    messageElement.className = `message ${isOutgoing ? 'out' : 'in'}`;
    messageElement.dataset.messageId = message.id;

    const timeString = new Date(message.sentTime).toLocaleTimeString([], {
        hour: '2-digit',
        minute: '2-digit'
    });

    messageElement.innerHTML = `
        <div class="message-content">${escapeHtml(message.content)}</div>
        <div class="message-time">
            ${timeString}
            ${isOutgoing ? '<span class="status-icon">✓</span>' : ''}
        </div>
    `;

    messagesList.appendChild(messageElement);
    messagesList.scrollTop = messagesList.scrollHeight;
}

function updateMessageStatus(messageId) {
    const messageElement = document.querySelector(`[data-message-id="${messageId}"]`);
    if (messageElement) {
        const statusIcon = messageElement.querySelector('.status-icon');
        if (statusIcon) {
            statusIcon.classList.add('read');
            statusIcon.textContent = '✓✓';
        }
    }
}

function showTypingIndicator(show) {if (typingIndicator) {
    typingIndicator.style.display = show ? 'block' : 'none';
}
}

```

```

    }
function updateUserStatus(status) {
    if (userStatus) {
        userStatus.innerHTML = status.isOnline
            ? '<span class="online">online</span>'
            : `<span>last seen ${new Date(status.lastOnline).toLocaleString()}</span>`;
    }
}

function escapeHtml(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}

async function sendMessage() {
    if (state.isSending) return;
    const message = messageInput.value.trim();
    if (!message) return;
    state.isSending = true;
    sendButton.disabled = true;

    try {
        await connection.invoke("SendMessage", parseInt(otherUserId), message);
        messageInput.value = "";
        messageInput.focus();
    } catch (err) {
        console.error("Send failed:", err);
        alert("Failed to send message: " + err.message);
    } finally {
        sendButton.disabled = false;
        state.isSending = false;
    }
}

```

```

let typingTimer;
messageInput.addEventListener('input', () => {
    clearTimeout(typingTimer);
    if (connection.state === signalR.HubConnectionState.Connected) {
        connection.invoke("SendTypingNotification", parseInt(otherUserId))
            .catch(err => console.error("Typing error:", err));
    }
    typingTimer = setTimeout(() => showTypingIndicator(false), 2000);
});

async function startConnection() {
    if (state.isConnecting || connection.state !==
signalR.HubConnectionState.Disconnected) {
        return;
    }
    state.isConnecting = true;
    state.connectionAttempts++;
    try {
        console.log("Attempting to connect...");
        await connection.start();
        console.log("Successfully connected to chat hub");
        await connection.invoke("JoinChatGroup", currentUserId, otherUserId);
        state.connectionAttempts = 0;
    } catch (err) {
        console.error("Connection error:", err);
        if (state.connectionAttempts < state.MAX_RETRY_ATTEMPTS) {
            setTimeout(startConnection, 5000);
        } else {
            alert("Failed to connect to chat. Please refresh the page.");
        }
    } finally {
        state.isConnecting = false;
    }
}

```

```

    }

    connection.onclose(async () => {
        console.log("Connection closed. Attempting to reconnect...");
        await startConnection();
    });

    sendButton.addEventListener('click', sendMessage);
    messageInput.addEventListener('keypress', (e) => {
        if (e.key === 'Enter') sendMessage();
    });

    startConnection();
});
</script>
}

```

Код из файла error_log_in.html

```

<h1>Ошибка</h1>
<p>Неверное имя пользователя или пароль</p>
<a href="/login">Войти</a>

```

Код из файла error_profile.html

```

<h1>Такого пользователя не существует</h1>

```

Код из файла error_sign_up.html

```

<h1>Ошибка</h1>
<p>Пользователь с таким именем уже существует</p>
<p>
    Обратно к <a href="/signup">регистрации</a>
</p>

```

Код из файла success_sign_up.html

```

<h3>Вы успешно зарегистрировались!</h3>
<p>
    <a href="/login">Войти</a>
</p>

```

Код из файла site.css

```

html {
    width: 100%;
    height: 100%;
}

body {
    display: flex;
    flex-flow: column nowrap;
    margin: 0px;
    background: silver;
}

```

```

    font-family: "Montserrat", sans-serif;
    font-size: 16px;
    min-height: 100vh;
    max-height: 100vh;
}

header {
    width: 100%;
    box-sizing: border-box;
    padding: 15px 10%;
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: #B2E0DF;
    color: #218583;
    border-bottom: 2px solid #218583;
}

header.logo {
    font-size: 25px;
}

header nav a {
    color: #218583;
    text-decoration: none;
    border: 0;
    margin-left: 15px;
}

header nav a:hover {
    text-decoration: underline;
}

main {
    display: block;
    flex-grow: 1;
    width: 80%;
    padding: 0px 10%;
    margin: 50px 0;
    position: relative;
}

footer {
    text-align: center;
    font-size: 25px;
    width: 100%;
    padding: 25px 0;
    color: #218583;
    background: #B2E0DF;
    border-top: 2px solid #218583;
}

```

```

form.login, form.signup {
  display: flex;
  flex-flow: column nowrap;
  align-items: center;
  justify-content: center;
}

label.login, label.signup {
  display: block;
  margin-top: 17px;
  font-size: 16px;
  width: 20%;
  text-align: center;
}

main a.login {
  color: black;
  text-decoration: none;
  border: 0;
  margin-top: 17px;
  font-size: 16px;
  width: 20%;
  text-align: center;
}

main a.login:hover {
  text-decoration: underline;
}

main input {
  box-sizing: border-box;
}

input.login, input.signup, input.search {
  width: 20%;
}

main input:not([type="submit"]) {
  display: block;
  border-radius: 15px;
  border: 0;
  padding: 10px;
  font-size: 16px;
}

input.login:not([type="submit"]), input.signup:not([type="submit"]) {
  margin-top: 6px;
}

main input:not([type="submit"]):focus {
  border: 0;
  outline: 0;
}

```



```

}

main input[type="submit"] {
  display: block;
  width: 20%;
  text-align: center;
  border-radius: 15px;
  border: 0;
  padding: 8px;
  background: #B2E0DF;
  font-size: 16px;
  color: #218583;
  cursor: pointer;
}

input.login[type="submit"], input.signup[type="submit"] {
  margin-top: 17px;
}

main input[type="submit"]:hover {
  background: #62ACAB;
}

main form span {
  color: #ff0000;
  font-weight: bold;
  display: block;
}

div.profile-info {
  max-width: 50%;
  margin: 15px 0 0 30px;
}

div.chats {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
}

div.inside-chats {
  width: 15%;
  margin: 20px;
  padding: 5px;
  display: block;
}

span.chats {
  display: block;
  width: 100%;
  font-weight: bold;
  font-size: 22px;
}

```

```

    text-align: center;
}

div.chats-list {
    margin-top: 15px;
}

div.chats-item {
    width: 100%;
}

ul.search-result-list {
    padding-inline-start: 0;
}

a.chats-item, a.search-result-item {
    display: block;
    border: none;
    border-radius: 15px;
    padding: 10px;
    margin: 5px 0px;
    background: darkgrey;
    color: black;
    text-decoration: none;
}

a.search-result-item {
    box-sizing: border-box;
    width: 20%;
    max-width: 50%;
}

a.chats-item:hover, a.search-result-item:hover {
    background: #81898D;
}

a.new-chat, a.to-all-chats {
    display: block;
    box-sizing: border-box;
    background: darkgrey;
    border: 2px solid grey;
    border-radius: 15px;
    padding: 8px;
    color: black;
    text-align: center;
    text-decoration: none;
}

a.new-chat:hover, a.to-all-chats:hover {
    background: #81898D;
}

```

```

a.to-all-chats {
  width: 20%;
}

a.profile {
  margin: 15px 0 0 30px;
}

div.chat {
  display: flex;
  flex-flow: column nowrap;
  position: absolute;
  width: 80%;
  height: 100%;
}

div.header {
  display: flex;
  justify-content: space-around;
  align-items: center;
  margin: 6px;
}

a.with-user {
  display: block;
  padding: 8px;
  color: black;
  text-align: center;
  text-decoration: none;
}

a.with-user:hover {
  text-decoration: underline;
}

div.messages-list {
  border-top: 2px solid grey;
  border-bottom: 2px solid grey;
  flex-grow: 1;
  overflow: auto;
  scroll-behavior: smooth;
}

div.messages-list::-webkit-scrollbar-thumb {
  background: grey;
}

form.send-message {
  display: flex;
  flex-flow: row nowrap;
}

```

```

input.send-message-text {
    width: 80%;
    margin-top: 6px;
    margin-right: 6px;
}

input.send-message-button{
    margin-top: 6px;
}

div.message {
    display: block;
    margin: 6px 0;
    padding: 6px;
    border: 2px solid grey;
    border-radius: 15px;
    width: fit-content;
    max-width: 60%;
}

div.out {
    margin-left: auto;
    border-bottom-right-radius: 0;
}

div.in {
    margin-right: auto;
    border-bottom-left-radius: 0;
}

div.message-text {
    white-space: break-spaces;
}

div.message-time {
    font-size: 12px;
    width: fit-content;
    margin-top: auto;
    margin-left: auto;
}

input.search {
    margin-top: 5px;
    margin-bottom: 5px;
}

button.search {
    display: block;
    width: 20%;
    text-align: center;
    border-radius: 15px;
    border: 0;
}

```

```
padding: 8px;
background: #B2E0DF;
font-size: 16px;
color: #218583;
cursor: pointer;
}
```

```
button.search:hover {
  background: #62ACAB;
}
```

```
span.no-msgs {
  display: block;
  text-align: center;
  margin: 10px;
}
```

Код из файла chat.js

```
window.addEventListener('load', () => scrollToBottom());
```

```
function scrollToBottom() {
  const div = document.getElementById('messages-list');
  div.scrollTop = div.scrollHeight;
}
```

```
async function fetchData() {
  const response = await fetch(window.location.href, {
    headers: {
      'Accept': 'text/html'
    },
    cache: 'no-store'
  });
}
```

```
const tempDiv = document.createElement('div');
tempDiv.innerHTML = await response.text();
```

```
const newDiv = tempDiv.querySelector("#messages-list");
document.getElementById('messages-list').innerHTML = newDiv.innerHTML;
scrollToBottom();
}
```

```
setInterval(fetchData, 5000);
```

Код из файла search_user.js

```
document.getElementById("search-button").onclick = function search() {
  var url = window.location.origin + window.location.pathname +
  `?username=${document.getElementById("username").value}`;
  window.location.href = url;
};
```

```
window.addEventListener('load', fillInput());
```

```
function fillInput() {
```

```

    const params = new URLSearchParams(window.location.search);
    document.getElementById("username").value = params.get('username');
}

```

Код из файла Register.cshtml

```

@model RegisterViewModel
<h2>Регистрация</h2>
<form asp-action="Register" method="post">
    <div asp-validation-summary="All" class="text-danger"></div>
    <div class="form-group">
        <label asp-for="DisplayName">Отображаемое имя</label>
        <input asp-for="DisplayName" class="form-control" />
        <span asp-validation-for="DisplayName" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Password">Пароль</label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="ConfirmPassword">Повторите пароль</label>
        <input asp-for="ConfirmPassword" class="form-control" />
        <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
    </div>
    <button type="submit" class="btn btn-primary">Зарегистрироваться</button>
</form>
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

Код из файла Login.cshtml

```

@model Chat.ViewModels.LoginViewModel
@{

```

```

    ViewData["Title"] = "Login";
}
<div class="row">
    <div class="col-md-4 offset-md-4">
        <h2>Вход</h2>
        <form asp-action="Login" method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Email" class="control-label"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Password" class="control-label">Пароль</label>
                <input asp-for="Password" type="password" class="form-control" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="form-group form-check">
                <input asp-for="RememberMe" class="form-check-input" />
                <label asp-for="RememberMe" class="form-check-label">Запомнить меня</label>
            </div>
            <div class="form-group">
                <button type="submit" class="btn btn-primary">Войти</button>
            </div>
        </form>
        <div class="mt-3">
            <a asp-action="Register" asp-controller="Account">Зарегистрируйтесь как новый
пользователь</a>
        </div>
    </div>
</div>
@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}

```

```
}
```

Код из файла User.cs

```
using Chat.Models;
using Microsoft.AspNetCore.Identity;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

public class User : IdentityUser<int>
{
    [Required]
    [MaxLength(50)]
    public string DisplayName { get; set; } = string.Empty;

    public DateTime RegistrationDate { get; set; } = DateTime.UtcNow;
    public DateTime LastOnline { get; set; } = DateTime.UtcNow;
    public bool IsOnline { get; set; }
    public string? AvatarUrl { get; set; }

    [NotMapped]
    public string? ConnectionId { get; set; }

    public virtual ICollection<Message> SentMessages { get; set; } = new
    HashSet<Message>();
    public virtual ICollection<Message> ReceivedMessages { get; set; } = new
    HashSet<Message>();
}
```

Код из файла Message.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Chat.Models
{
    public class Message
    {
        public int Id { get; set; }
        [ForeignKey("Sender")]
        public int SenderId { get; set; }
        [ForeignKey("Receiver")]
        public int ReceiverId { get; set; }
        [Required]
        public string Content { get; set; } = string.Empty;
        public DateTime SentTime { get; set; }
        public bool IsRead { get; set; }
        public DateTime? ReadTime { get; set; }
        public virtual User Sender { get; set; } = null!;
        public virtual User Receiver { get; set; } = null!;
    }
}
```


Код из файла RegisterViewModel.cs

```
using System.ComponentModel.DataAnnotations;

public class RegisterViewModel
{
    [Required]
    [Display(Name = "Display Name")]
    [MaxLength(50)]
    public string DisplayName { get; set; } = string.Empty;
    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;
    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;
    [DataType(DataType.Password)]
    [Display(Name = "Confirm Passwor")]
    [Compare("Password", ErrorMessage = "Пароль не совпадает.")]
    public string ConfirmPassword { get; set; } = string.Empty;
}
```

Код из файла LoginViewModel.cs

```
using System.ComponentModel.DataAnnotations;
namespace Chat.ViewModels
{
    public class LoginViewModel
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        [DataType(DataType.Password)]
        public string Password { get; set; } = string.Empty;

        [Display(Name = "Remember me")]
        public bool RememberMe { get; set; }
    }
}
```

Код из файла ChatViewModel.cs

```
using Chat.Models;
using System.Collections.Generic;

public class ChatViewModel
{
    public User CurrentUser { get; set; } = null!; // текущий пользователь
    public User OtherUser { get; set; } = null!; // собеседник
    public List<Message> Messages { get; set; } = new List<Message>(); // история сообщений
}
```

```

    public string NewMessage { get; set; } = string.Empty; //новое сообщение
}

```

Код из файла ChatHub.cs

```

using Chat.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.SignalR;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
using System;
using System.Threading.Tasks;
[Authorize]
public class ChatHub : Hub
{
    private readonly ApplicationDbContext _context;
    private readonly ILogger<ChatHub> _logger;

    public ChatHub(ApplicationDbContext context, ILogger<ChatHub> logger)
    {
        _context = context;
        _logger = logger;
    }

    public async Task JoinChatGroup(string currentUserId, string otherUserId)
    {
        try
        {
            // Создаем универсальное имя группы (сортировка ID)
            var groupName = GetGroupName(currentUserId, otherUserId);
            await Groups.AddToGroupAsync(Context.ConnectionId, groupName);

            _logger.LogInformation("User {UserId} joined group {GroupName}", currentUserId,
            groupName);

            var messages = await _context.Messages
                .Where(m => (m.SenderId == int.Parse(currentUserId) && m.ReceiverId ==
            int.Parse(otherUserId)) ||

```

```

        (m.SenderId == int.Parse(otherUserId) && m.ReceiverId ==
int.Parse(currentUserId)))
        .OrderBy(m => m.SentTime)
        .ToListAsync();
foreach (var message in messages)
{
    await Clients.Caller.SendAsync("ReceiveMessage", new
    {
        Id = message.Id,
        SenderId = message.SenderId,
        Content = message.Content,
        SentTime = message.SentTime,
        IsRead = message.IsRead
    });
}
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error joining chat group");
    throw new HubException("Failed to join chat group");
}
}
public async Task SendMessage(int receiverId, string messageContent)
{
    try
    {
        var senderId = int.Parse(Context.UserIdentifier);
        if (string.IsNullOrEmpty(messageContent))
            throw new HubException("Message cannot be empty");
        var message = new Message
        {
            SenderId = senderId,
            ReceiverId = receiverId,
            Content = messageContent,
            SentTime = DateTime.UtcNow,

```

```

        IsRead = false
    };
    _context.Messages.Add(message);
    await _context.SaveChangesAsync();
    var groupName = GetGroupName(senderId.ToString(), receiverId.ToString());
    await Clients.Group(groupName).SendAsync("ReceiveMessage", new
    {
        Id = message.Id,
        SenderId = senderId,
        Content = messageContent,
        SentTime = message.SentTime,
        IsRead = false
    });
    await Clients.Caller.SendAsync("MessageSent", new
    {
        Id = message.Id,
        ReceiverId = receiverId,
        Content = messageContent,
        SentTime = message.SentTime
    });
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error sending message");
    throw new HubException("Error sending message");
}
}

private string GetGroupName(string userId1, string userId2)
{
    // Универсальное имя группы (сортировка ID для одинакового имени)
    return string.Compare(userId1, userId2) < 0
        ? $"chat_{userId1}_{userId2}"
        : $"chat_{userId2}_{userId1}";
}

```

```

public async Task SendTypingNotification(int receiverId)
{
    try
    {
        var senderId = int.Parse(Context.UserIdentifier);
        await Clients.User(receiverId.ToString()).SendAsync("UserTyping");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error sending typing notification");
    }
}

```

```

public override async Task OnConnectedAsync()
{
    var userId = Context.UserIdentifier;
    var user = await _context.Users.FindAsync(int.Parse(userId));
    if (user != null)
    {
        user.IsOnline = true;
        await _context.SaveChangesAsync();

        await Clients.Others.SendAsync("UserStatusChanged", new
        {
            UserId = userId,
            IsOnline = true,
            LastOnline = user.LastOnline
        });
    }
    await base.OnConnectedAsync();
}

```

```

public override async Task OnDisconnectedAsync(Exception? exception)
{

```

```

var userId = Context.UserIdentifier;
var user = await _context.Users.FindAsync(int.Parse(userId));
if (user != null)
{
    user.IsOnline = false;
    user.LastOnline = DateTime.UtcNow;
    await _context.SaveChangesAsync();
    await Clients.Others.SendAsync("UserStatusChanged", new
    {
        UserId = userId,
        IsOnline = false,
        LastOnline = user.LastOnline
    });
}
await base.OnDisconnectedAsync(exception);
}
}

```

Код из файла SeedData.cs

```

using Chat.Data;
using Microsoft.AspNetCore.Identity;
namespace Chat.Data
{
    public static class SeedData
    {
        public static async Task Initialize(
            IServiceProvider serviceProvider,
            UserManager<User> userManager,
            RoleManager<IdentityRole<int>> roleManager)
        {
            using (var context = serviceProvider.GetRequiredService<AppDbContext>())
            {
                // Ваша логика инициализации данных
                await SeedRoles(roleManager);
                await SeedUsers(userManager);
            }
        }

        private static async Task SeedRoles(RoleManager<IdentityRole<int>> roleManager)
        {
            string[] roleNames = { "Admin", "User" };

            foreach (var roleName in roleNames)
            {

```



```

        .WithMany(u => u.ReceivedMessages) // Указываем навигационное свойство в User
        .HasForeignKey(m => m.ReceiverId)
        .OnDelete(DeleteBehavior.Restrict)
        .IsRequired(); // Явно указываем, что получатель обязателен
modelBuilder.Entity<Message>()
    .Property(m => m.SentTime)
    .HasDefaultValueSql("GETDATE()"); // Автоматическое установление времени
отправки
    }
}
}

```

Код из файла UsersController.cs

```

using Chat.ViewModels;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Chat.Models; // Добавьте эту строку для доступа к моделям
namespace Chat.Controllers
{
    public class UsersController : Controller
    {
        private readonly ApplicationDbContext _context;
        public UsersController(ApplicationDbContext context)
        {
            _context = context;
        }

        [Authorize]
        public async Task<IActionResult> Index()
        {
            var users = await _context.Users
                .OrderByDescending(u => u.IsOnline)
                .ThenBy(u => u.UserName) // Изменили Username на UserName (из IdentityUser)
                .ToListAsync();

            return View(users);
        }

        [Authorize]
        public async Task<IActionResult> Chat(int id)
        {
            var currentUserIdString = User.FindFirstValue(ClaimTypes.NameIdentifier);
            if (string.IsNullOrEmpty(currentUserIdString))
            {
                return Challenge();
            }
            var currentUserId = int.Parse(currentUserIdString);
            var messages = await _context.Messages
                .Where(m => (m.SenderId == currentUserId && m.ReceiverId == id) ||
                    (m.SenderId == id && m.ReceiverId == currentUserId))
                .OrderBy(m => m.SentTime)
                .ToListAsync();
        }
    }
}

```



```

        var currentUser = await _context.Users.FindAsync(currentUserId);
        var otherUser = await _context.Users.FindAsync(id);
        if (currentUser == null || otherUser == null)
        {
            return NotFound();
        }
        var viewModel = new ChatViewModel
        {
            CurrentUser = currentUser,
            OtherUser = otherUser,
            Messages = messages
        };
        return View(viewModel);
    }
}

```

Код из файла ChatController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
[Authorize]
public class ChatController : Controller
{
    private readonly ApplicationDbContext _context;
    private readonly UserManager<User> _userManager;

    public ChatController(ApplicationDbContext context, UserManager<User> userManager)
    {
        _context = context;
        _userManager = userManager;
    }
    [HttpGet("Chat/Conversation/{userId:int}")] // Явно указываем тип параметра
    public async Task<IActionResult> Conversation(int userId)
    {
        try
        {
            var currentUser = await _userManager.GetUserAsync(User);
            if (currentUser == null) return Challenge(); // Перенаправит на страницу входа
            var otherUser = await _context.Users
                .AsNoTracking()
                .FirstOrDefaultAsync(u => u.Id == userId);
            if (otherUser == null) return NotFound();
            var messages = await _context.Messages
                .Where(m => (m.SenderId == currentUser.Id && m.ReceiverId == userId) ||
                    (m.SenderId == userId && m.ReceiverId == currentUser.Id))
                .OrderBy(m => m.SentTime)
                .AsNoTracking()
                .ToListAsync();
            return View(new ChatViewModel
            {

```

```

        CurrentUser = currentUser,
        OtherUser = otherUser,
        Messages = messages
    });
}
catch (Exception ex)
{
    // Логирование ошибки
    return StatusCode(500, "Внутренняя ошибка сервера");
}
}
[HttpGet("Chat")]
public async Task<IActionResult> Index()
{
    var currentUser = await _userManager.GetUserAsync(User);
    if (currentUser == null) return Challenge();

    var users = await _context.Users
        .Where(u => u.Id != currentUser.Id)
        .OrderByDescending(u => u.IsOnline)
        .ThenBy(u => u.UserName)
        .AsNoTracking()
        .ToListAsync();

    return View(users);
}
}

```

Код из файла AccountController.cs

```

using Chat.ViewModels;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
public class AccountController : Controller
{
    private readonly UserManager<User> _userManager;
    private readonly SignInManager<User> _signInManager;
    private readonly ILogger<AccountController> _logger;
    public AccountController(
        UserManager<User> userManager,
        SignInManager<User> signInManager,
        ILogger<AccountController> logger)
    {
        _userManager = userManager;
        _signInManager = signInManager;
        _logger = logger;
    }
    [HttpGet]
    public IActionResult Register()
    {
        return View();
    }
    [ValidateAntiForgeryToken]

```

```

[HttpPost]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new User
        {
            UserName = model.Email,
            Email = model.Email,
            DisplayName = model.DisplayName
        };
        var result = await _userManager.CreateAsync(user, model.Password);
        if (result.Succeeded)
        {
            _logger.LogInformation("Пользователь создал новую учетную запись с паролем.");

            await _signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("Index", "Home");
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }
    }

    return View(model);
}

[HttpGet]
public IActionResult Login(string? returnUrl = null)
{
    ViewData["ReturnUrl"] = returnUrl;
    return View();
}

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Login(LoginViewModel model, string? returnUrl = null)
{
    if (ModelState.IsValid)
    {
        var result = await _signInManager.PasswordSignInAsync(
            model.Email, model.Password, model.RememberMe, lockoutOnFailure: false);

        if (result.Succeeded)
        {
            _logger.LogInformation("Пользователь вошел в систему.");
            return RedirectToAction("Index", "Home");
        }

        ModelState.AddModelError(string.Empty, "Неудачная попытка входа в систему.");
    }
}

```

```

    return View(model);
}

[HttpPost]
public async Task<IActionResult> Logout()
{
    await _signInManager.SignOutAsync();
    _logger.LogInformation("Пользователь вышел из системы.");
    return RedirectToAction("Index", "Home");
}
}

Код из файла site.js
document.addEventListener('DOMContentLoaded', function () {
    // Состояние приложения
    const state = {
        isConnected: false,
        isSending: false,
        connectionAttempts: 0,
        MAX_RETRY_ATTEMPTS: 5,
        lastSentMessage: "",
        lastSentTime: 0,
        activeGroup: null
    };
    const chatContainer = document.querySelector('.vk-style-chat');
    if (!chatContainer) {
        console.error("Chat container not found");
        return;
    }
    const currentUserId = chatContainer.dataset.currentUserId;
    const otherUserId = chatContainer.dataset.otherUserId;
    const messageInput = document.getElementById('messageInput');
    const sendButton = document.getElementById('sendButton');
    const messagesList = document.getElementById('messagesList');
    const userStatus = document.getElementById('userStatus');
    const typingIndicator = document.getElementById('typingIndicator');
    const connection = new signalR.HubConnectionBuilder()
        .withUrl("/chatHub", {
            skipNegotiation: true,
            transport: signalR.HttpTransportType.WebSockets
        })
        .configureLogging(signalR.LogLevel.Warning)
        .withAutomaticReconnect({
            nextRetryDelayInMilliseconds: retryContext => {
                if (retryContext.elapsedMilliseconds < 60000) {
                    return Math.random() * 2000 + 2000;
                }
                return null;
            }
        })
        .build();
    connection.on("ReceiveMessage", (message) => {
        console.log("Received message:", message);
    });

```

```

    if (!isMessageExists(message.id)) {
        addMessage(message, message.senderId === currentUserId);
    }
});
connection.on("MessageSent", (message) => {
    console.log("Message sent confirmation:", message);
    updateMessageStatus(message.id);
});
connection.on("UserTyping", () => {
    console.log("User is typing...");
    showTypingIndicator(true);
});
connection.on("UserStatusChanged", (status) => {
    console.log("User status changed:", status);
    updateUserStatus(status);
});
function isMessageExists(messageId) {
    return !!document.querySelector(`[data-message-id="${messageId}"]`);
}

function addMessage(message, isOutgoing) {
    const messageElement = document.createElement('div');
    messageElement.className = `message ${isOutgoing ? 'out' : 'in'}`;
    messageElement.dataset.messageId = message.id;
    const timeString = new Date(message.sentTime).toLocaleTimeString([], {
        hour: '2-digit',
        minute: '2-digit'
    });
    messageElement.innerHTML = `
        <div class="message-content">${escapeHtml(message.content)}</div>
        <div class="message-time">
            ${timeString}
            ${isOutgoing ? '<span class="status-icon">✓</span>' : ''}
        </div>
    `;
    messagesList.appendChild(messageElement);
    messagesList.scrollTop = messagesList.scrollHeight;
}

function updateMessageStatus(messageId) {
    const messageElement = document.querySelector(`[data-message-id="${messageId}"]`);
    if (messageElement) {
        const statusIcon = messageElement.querySelector('.status-icon');
        if (statusIcon) {
            statusIcon.classList.add('read');
            statusIcon.textContent = '✓✓';
        }
    }
}

function showTypingIndicator(show) {
    if (typingIndicator) {
        typingIndicator.style.display = show ? 'block' : 'none';
    }
}

```

```

        if (show) {
            setTimeout(() => showTypingIndicator(false), 2000);
        }
    }
}

function updateUserStatus(status) {
    if (userStatus) {
        userStatus.innerHTML = status.isOnline
            ? '<span class="online">online</span>'
            : '<span>last seen ${new Date(status.lastOnline).toLocaleString()}</span>';
    }
}

function escapeHtml(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}

async function sendMessage() {
    if (state.isSending || !connection.connectionStarted) return;
    const message = messageInput.value.trim();
    if (!message) return;
    const now = Date.now();
    if (message === state.lastSentMessage && (now - state.lastSentTime) < 2000) {
        return;
    }
    state.lastSentMessage = message;
    state.lastSentTime = now;
    state.isSending = true;
    sendButton.disabled = true;
    try {
        console.log("Sending message to:", otherUserId);
        await connection.invoke("SendMessage", parseInt(otherUserId), message);
        messageInput.value = "";
        messageInput.focus();
    } catch (err) {
        console.error("Send failed:", err);
        alert("Failed to send message: " + err.message);
    } finally {
        sendButton.disabled = false;
        state.isSending = false;
    }
}

let typingTimer;
messageInput.addEventListener('input', () => {
    clearTimeout(typingTimer);
    if (connection.state === signalR.HubConnectionState.Connected) {
        console.log("Sending typing notification");
        connection.invoke("SendTypingNotification", parseInt(otherUserId))
            .catch(err => console.error("Typing error:", err));
    }
});

```

```

    }
    typingTimer = setTimeout(() => showTypingIndicator(false), 2000);
  });

// Подключение к группе чата
async function joinChatGroup() {
  try {
    console.log("Joining chat group for users:", currentUserId, otherUserId);
    await connection.invoke("JoinChatGroup", currentUserId, otherUserId);
    state.activeGroup = `chat_${currentUserId}_${otherUserId}`;
    console.log("Successfully joined chat group");
  } catch (err) {
    console.error("Error joining chat group:", err);
    setTimeout(joinChatGroup, 5000);
  }
}

async function startConnection() {
  if (state.isConnecting || connection.state !== signalR.HubConnectionState.Disconnected) {
    return;
  }
  state.isConnecting = true;
  state.connectionAttempts++;
  try {
    console.log("Attempting to connect to SignalR hub...");
    await connection.start();
    console.log("Successfully connected to chat hub");
    await joinChatGroup();
    state.connectionAttempts = 0;
  } catch (err) {
    console.error("Connection error:", err);
    if (state.connectionAttempts < state.MAX_RETRY_ATTEMPTS) {
      setTimeout(startConnection, 5000);
    } else {
      alert("Failed to connect to chat. Please refresh the page.");
    }
  } finally {
    state.isConnecting = false;
  }
}

connection.onclose(async (error) => {
  console.log("Connection closed. Error:", error);
  await startConnection();
});

sendMessageButton.addEventListener('click', sendMessage);
messageInput.addEventListener('keypress', (e) => {
  if (e.key === 'Enter') sendMessage();
});

startConnection();
setInterval(() => {
  if (connection.state === signalR.HubConnectionState.Connected) {
    console.log("Connection state: Connected");
  }
}, 1000);

```

```

    } else {
        console.log("Connection state:", connection.state);
    }
}, 10000);
});

```

Код из файла site.css

```

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f5f5f5;
}
.chat-container {
    background-color: white;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
    padding: 20px;
    margin-top: 20px;
}

.chat-messages {
    height: 500px;
    overflow-y: auto;
    padding: 10px;
    border: 1px solid #e0e0e0;
    border-radius: 5px;
    margin-bottom: 15px;
    background-color: #fafafa;
}

.message {
    margin-bottom: 12px;
    padding: 10px 15px;
    border-radius: 18px;
    max-width: 70%;
    word-wrap: break-word;
}

.message.sent {
    background-color: #007bff;
    color: white;
    margin-left: auto;
    border-bottom-right-radius: 5px;
}

.message.received {
    background-color: #e9ecef;
    margin-right: auto;
    border-bottom-left-radius: 5px;
}

.message-content {
    margin-bottom: 5px;
}

.message-status {
    margin-left: 8px;
    font-size: 0.8em;
    opacity: 0.7;
}

```



```

    }
    .message-status.read {
        color: #4CAF50;
    }
    .message.received {
        background-color: #f1f1f1;
        margin-right: auto;
        margin-left: 0;
    }
    .message.sent {
        background-color: #007bff;
        color: white;
        margin-left: auto;
        margin-right: 0;
    }
    .message-time {
        font-size: 0.75rem;
        opacity: 0.8;
        text-align: right;
    }
    .chat-input {
        display: flex;
        margin-top: 10px;
    }
    .chat-input input {
        flex-grow: 1;
        border-radius: 20px;
        padding: 10px 15px;
        border: 1px solid #ddd;
        margin-right: 10px;
    }
    .chat-input button {
        border-radius: 20px;
        padding: 10px 20px;
    }
    @keyframes fadeIn {
        from {
            opacity: 0;
            transform: translateY(10px);
        }

        to {
            opacity: 1;
            transform: translateY(0);
        }
    }
    .message {
        animation: fadeIn 0.3s ease-out;
    }
    .user-list {
        max-height: 600px;
        overflow-y: auto;
    }

```

```
}

.user-item {
  transition: all 0.2s;
}

.user-item:hover {
  background-color: #f8f9fa;
  transform: translateX(5px);
}

.online-badge {
  width: 10px;
  height: 10px;
  border-radius: 50%;
  display: inline-block;
  margin-right: 5px;
}

.online {
  background-color: #28a745;
}

.offline {
  background-color: #6c757d;
}
```

Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ижевский государственный технический университет имени М.Т. Калашникова»
Факультет «Информационные технологии»
Рабочий график (план) практики

Обучающийся _____
ФИО

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

шифр наименование

направленность (профиль) подготовки «Организация и программирование
вычислительных и информационных систем»

Курс _____

Форма обучения _____

Факультет ИТ группа _____

Вид, тип, способ прохождения практики – учебная (ознакомительная) практика,
стационарная (в г. Ижевск)/выездная

Срок прохождения практики с _____ по _____

Организация (название), город _____

Руководитель практики от Университета

Вдовин Алексей Юрьевич, доцент кафедры «Вычислительная техника»

ФИО полностью, должность

Ответственное лицо от профильной организации за организацию практики

ФИО полностью, должность

Индивидуальное задание на практику: _____

Рабочий график (план) практики

Содержание практики (содержание работ)	Срок выполнения	Планируемые результаты
1. Инструктаж по ТБ		-
2. Составление ТЗ и согласование его с руководителем		ТЗ
3. Изучение предметной области, среды разработки и пр.		Получение требуемых знаний в предметной области
4. Разработка основных алгоритмов программы		Алгоритмы программы
5. Разработка интерфейса		Макет интерфейса программы
6. Реализация программы		Программа
7. Подготовка отчета	23.06.2025-26.06.2025	Отчет

Печатается с оборотной стороны рабочего графика (плана) практики

Ознакомлен с Программой практики и Положением о практике обучающихся
ИжГТУ имени М.Т. Калашникова

ФИО обучающегося, подпись, дата

Проведен инструктаж обучающегося технике безопасности, пожарной безопасности,
требованиям охраны труда, ознакомление с правилами внутреннего распорядка

ФИО инструктирующего от профильной организации, должность, подпись, дата

Индивидуальное задание, содержание и планируемые результаты практики согласованы

/ « » 202
подпись ответственного лица от профильной организации за организацию практики, расшифровка подписи

/ « » 202
подпись руководителя практики от Университета, расшифровка подписи

Задание принял к исполнению: / « » 20
подпись обучающегося расшифровка подписи